# How make your own texture

First, we need some preconditions. That would be having a template for which we want to make the custom texture.

Let us copy the template of vanilla *krug_scavenger* and create a *temp_scavenger* template, following the guidelines for making your own template.

Now, let us focus on making the custom texture.

# 1. Identify the texture

We do this by looking at the [aspect] block in the template. Then, we look at the [textures] block. Whatever is there, are our textures. In the *krug_scavenger* template, we can easily spot this. If you see textures defined differently, or not defined at all, do not despair – check the Final Remarks section at the end of this document.

```
[t:template,n:temp_scavenger]
{
	category_name = "1W_evil_a";
	doc = "temp scavenger";
	specializes = base_krug;
	[actor]
	{
		[skills]
		{
			dexterity = 4, 0;
			intelligence = 3, 0;
			strength = 5, 0;
		}
	}
	[aspect]
	{
		experience_value = 4;
		life = 8;
		mana = 6;
		max_life = 8;
		max_mana = 6;
		scale_base = 0.9;
		selection_indicator_scale = 0.85;
		[textures]
		{
			0 = b_c_eam_ksv;
		}
	}
	[attack]
	{
		damage_max = 2;
		damage_min = 1;
	}
```

## 2. Find the texture file

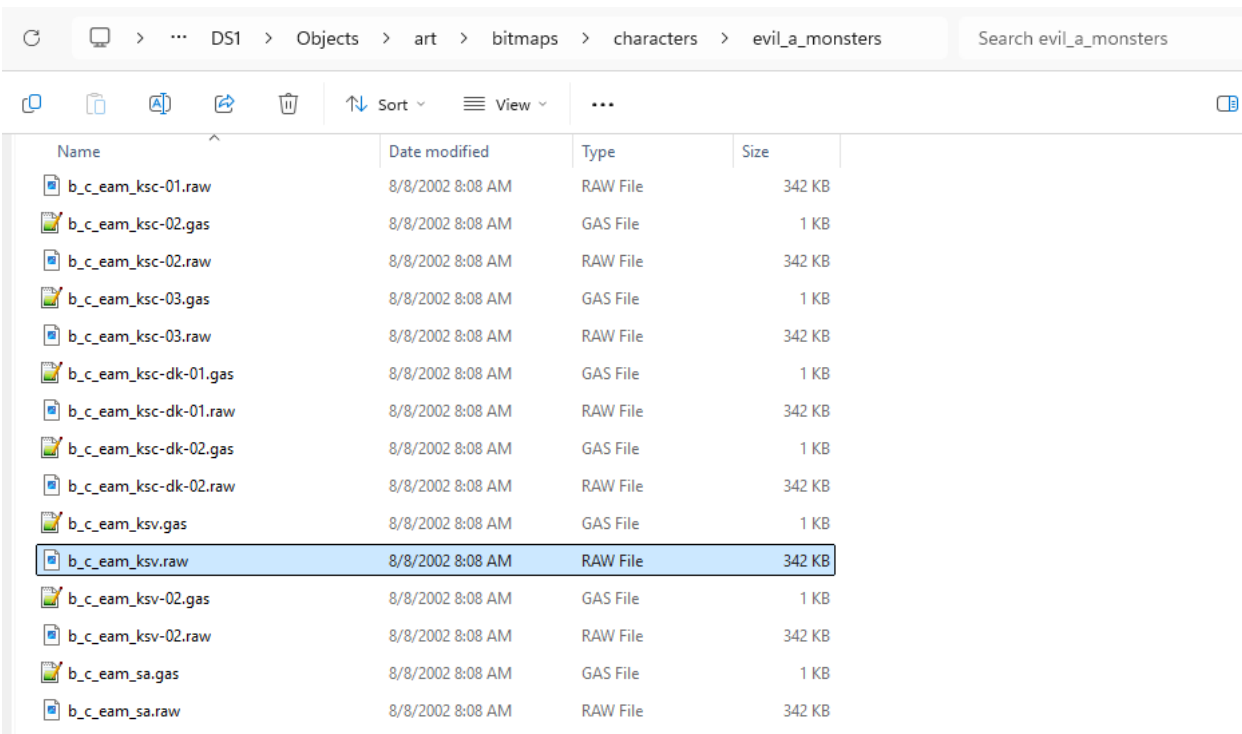
For this, we need the *Objects.dsres* file untanked via *TankViewer* on our disk. If in doubt, check guidelines for making your own template.

Now, the texture name in the template gives us the breadcrumbs to follow to find the texture (alternatively, the Visual Studio Code can help).

In this case, the b_c_eam_ksv texture would translate into path relative to the untanked Objects.dsres folder:

art/bitmaps/characters/evil_a_monsters/b_c_eam_ksv.raw

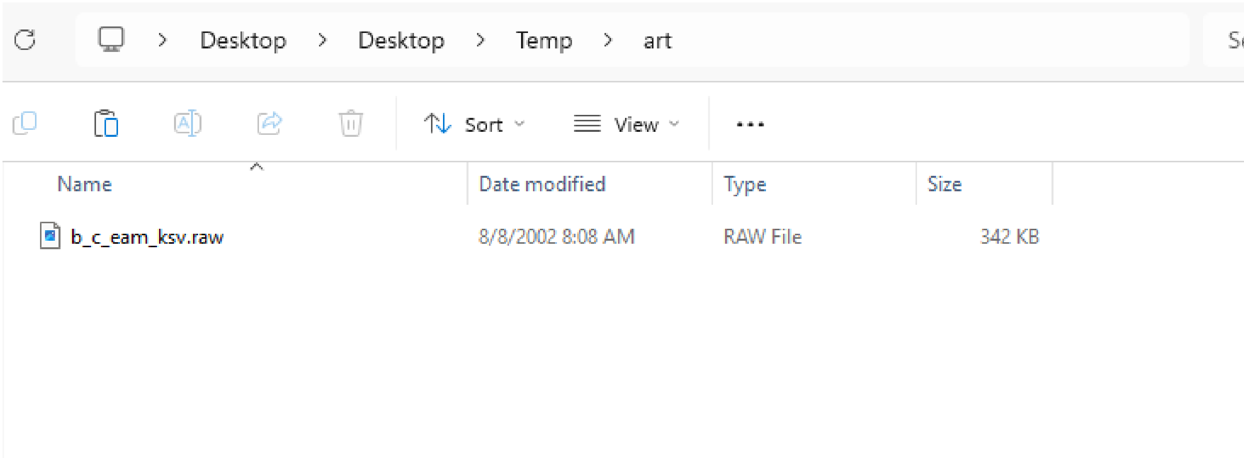
All textures are in the art subfolder, then the folder structure indicated by the underscore-separated abbreviations, and finaly, the texture name is also filename of the texture file, without extension.



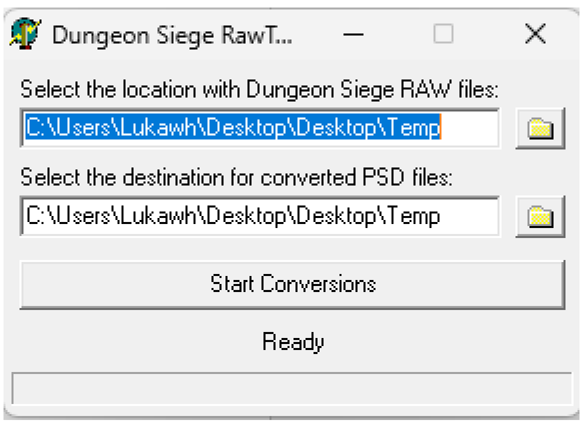

## 3. Convert to psd

Now, the .raw format of the textures are proprietary to Dungeon Siege (do not confuse it with generally-known raw format). This also means we need to convert it with proprietary software. The RawToPsdGUI.exe will do the job.

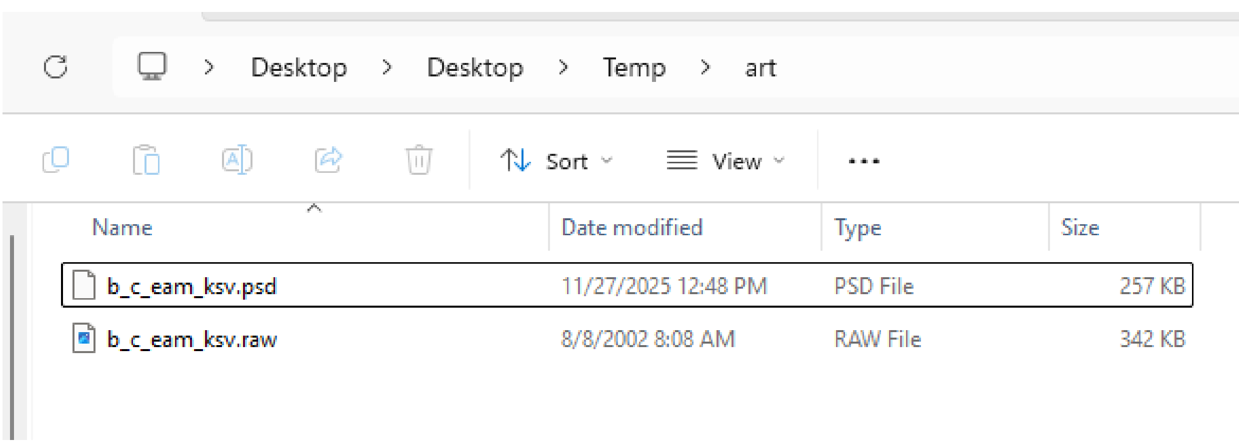
We can copy the desired texture into some temporary folder like so:




Then we run the RawToPsdGUI.exe, select the path to our temporary folder, select the path to output folder (it can be the same folder), and hit Start Conversions. Note that the tool will recursively search all subdirectories and perform conversion there, too.




Finally, we have the PSD file of the vanilla texture ready.

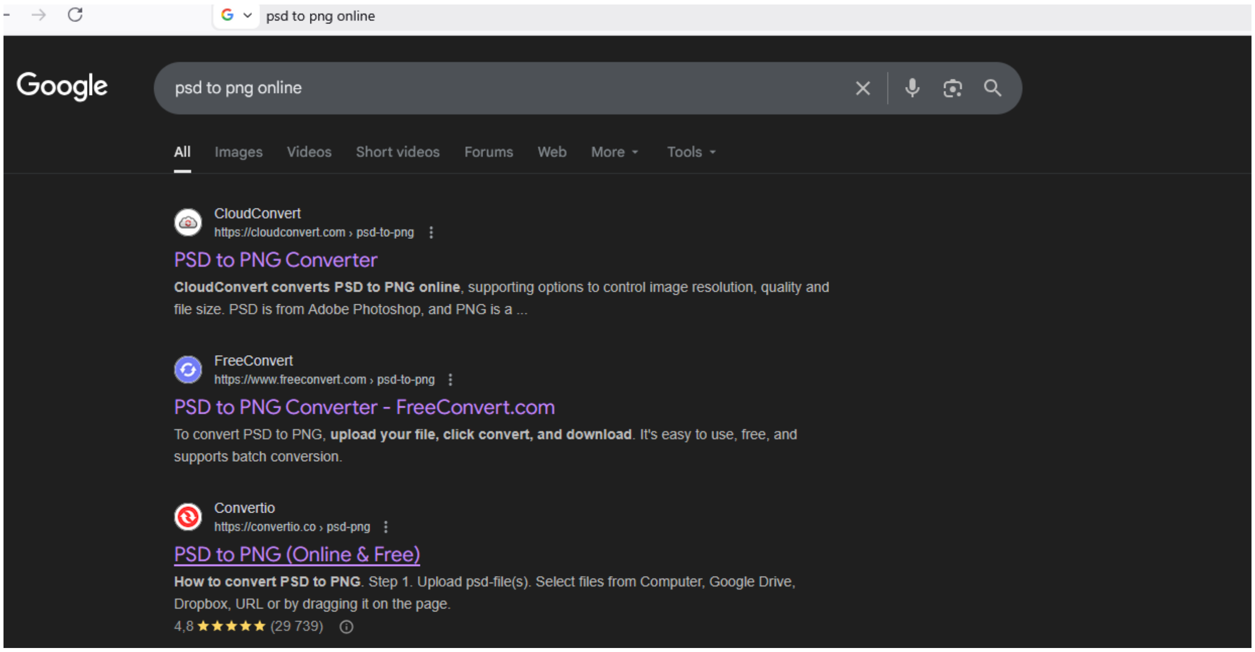


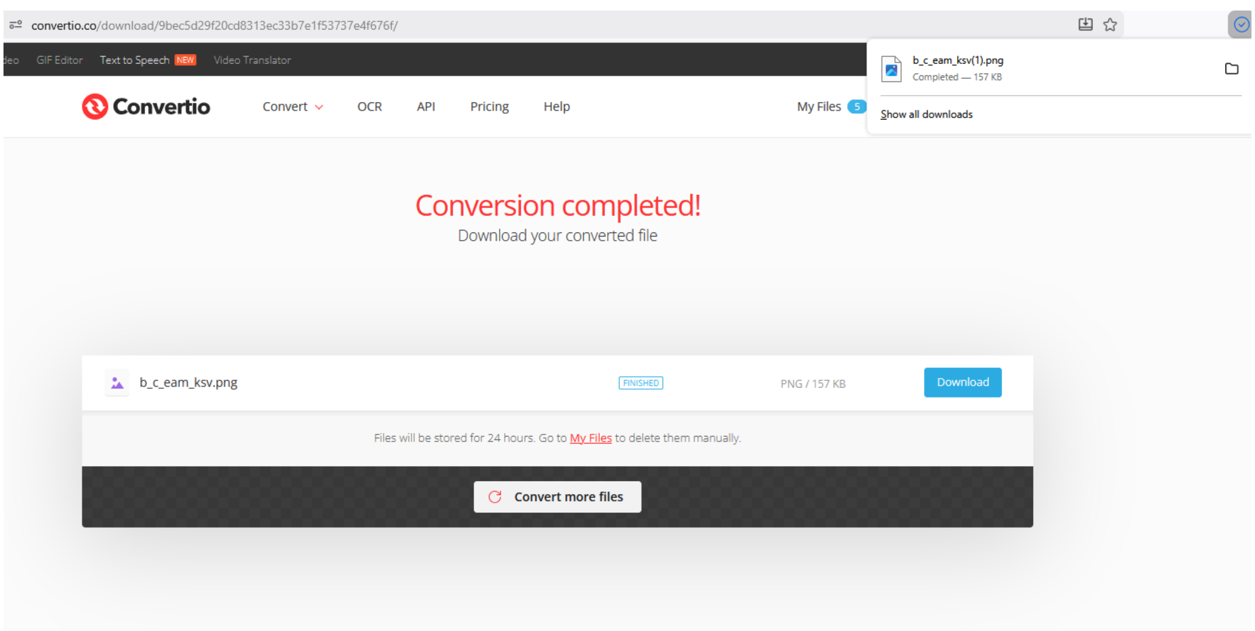

Alternatively, we can convert the entire untanked Objects.dsres folder hierarchy, and have PSD files ready for any further work.

## 4. Convert to png

Those who intend to use drawing software that can work PSD files can skip this step.

If your software requires other formats on input, you should convert to PNG (recommended). Personally, I like to use questionable free online converters. So, however you do it, convert your PSD file to a PNG file.



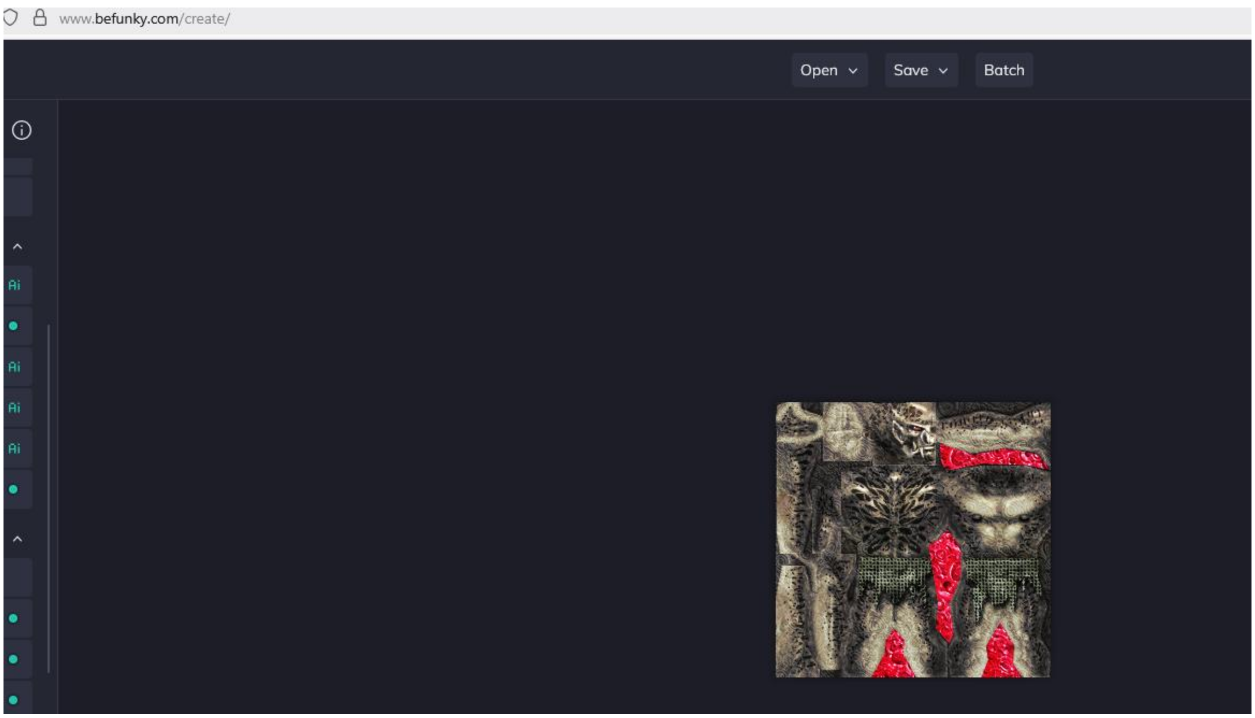
## 5. Modify the texture

Now that we have the texture in either the PSD or PNG format, we can modify the texture to our liking. Some textures are easy to understand (what is head, what are limbs, what is torso), others might be tougher.

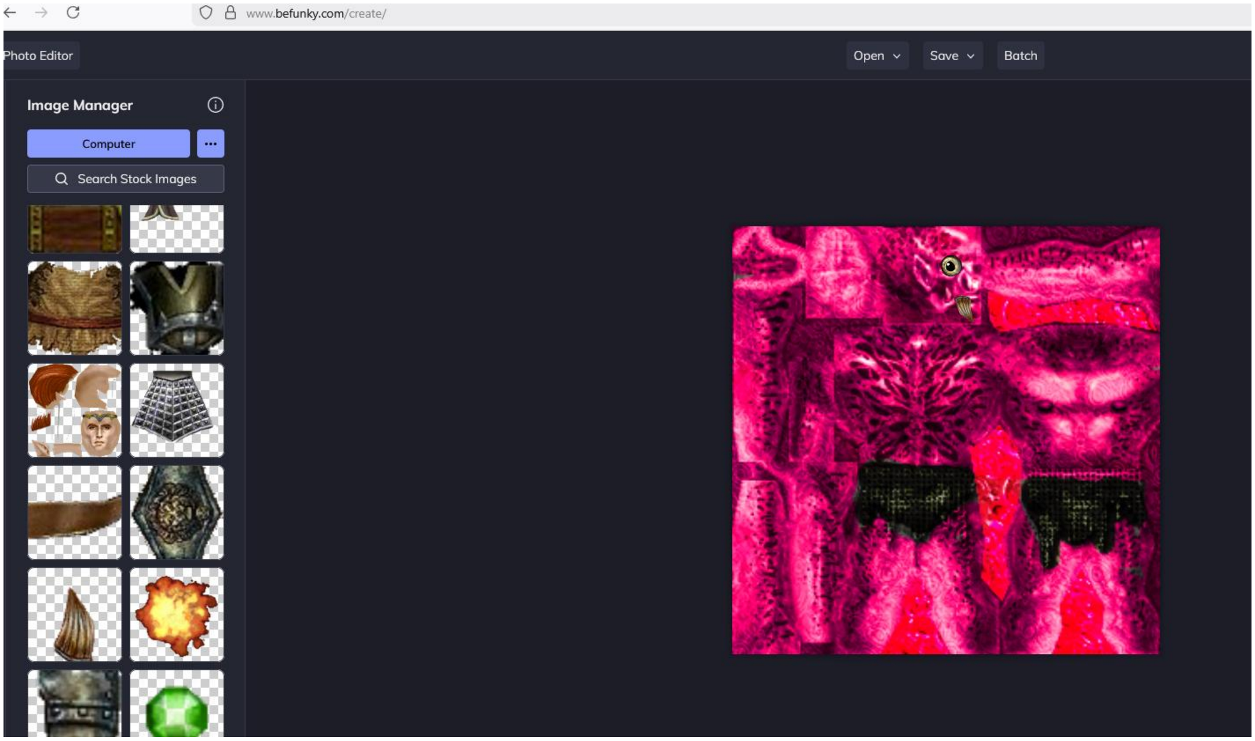
You can use any drawing software but take care not to butcher the format or layers of the texture, as the result can be unpredictable. Myself, I use silly online drawing tool BeFunky.

I am not a great artist, but I find this barbie-style guy quite dashing.

Before:




After:

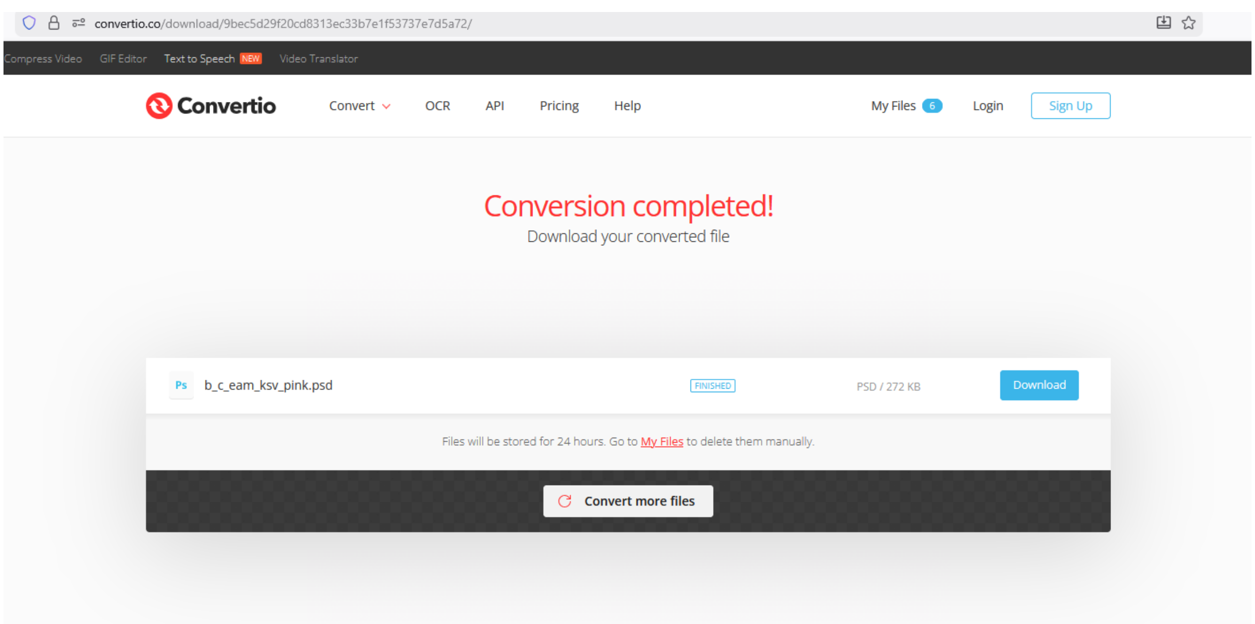



We can save our texture as b_c_eam_ksv_pink.png.

## 6. Convert to psd

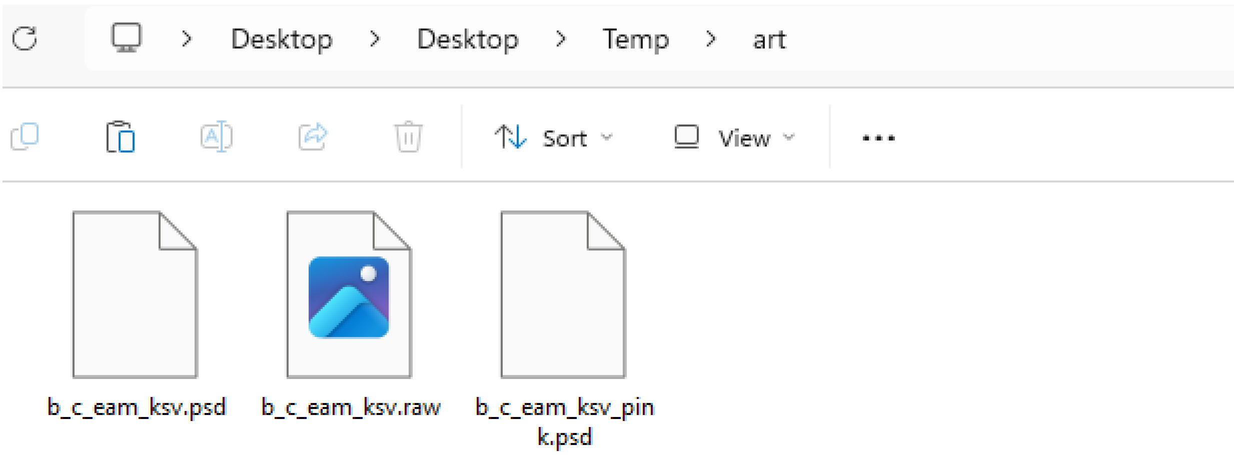
If you use a drawing tool that can work PSD files, you may happily skip this step again.

Those who don't, need to convert their PNG file to PSD format, possibly with the same tool as in step 4.

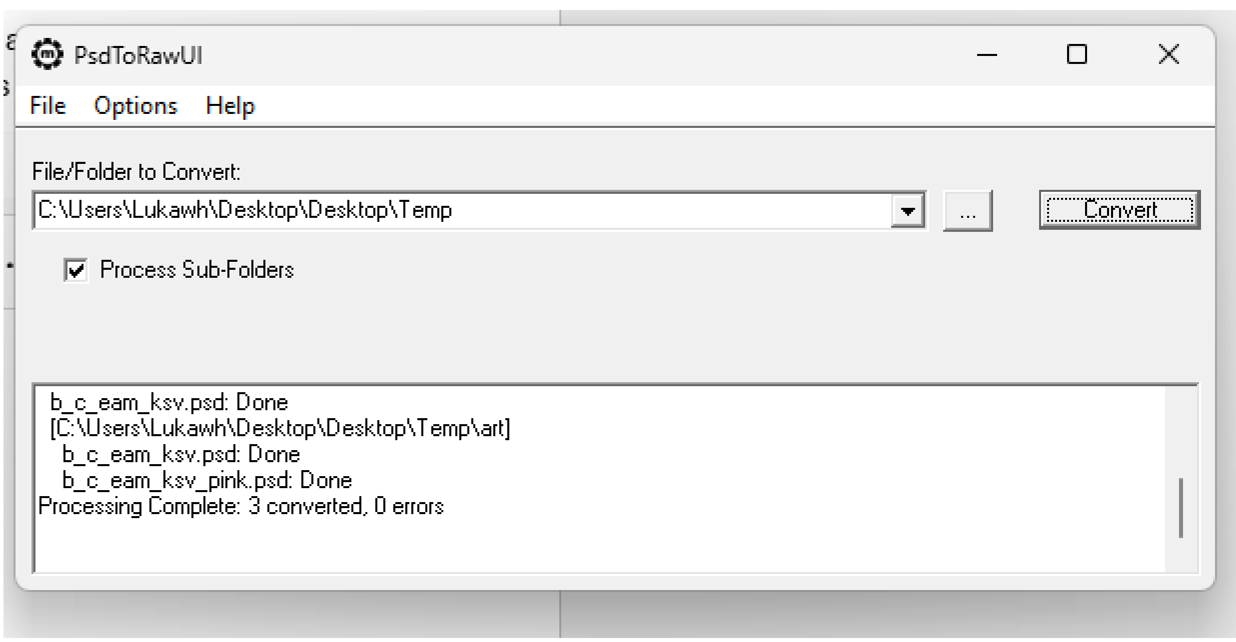


## 7. Convert to raw

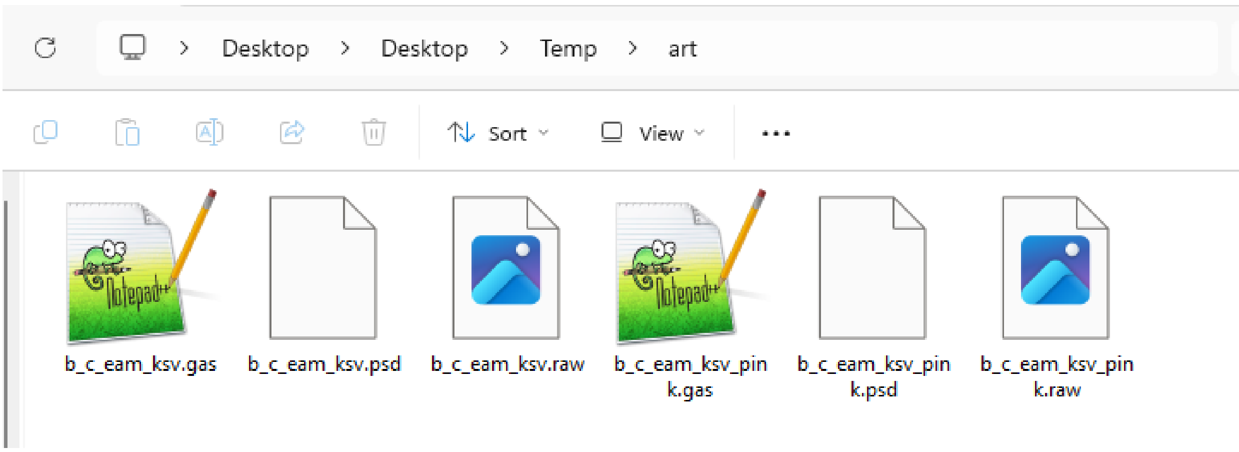
Now we need to convert the PSD file to the proprietary RAW format. PsdToRawUI.exe is perfect for this. Again, a temporary folder will serve us as good as it did in step 3.




The input folder of PsdToRawUI.exe is also its output folder.

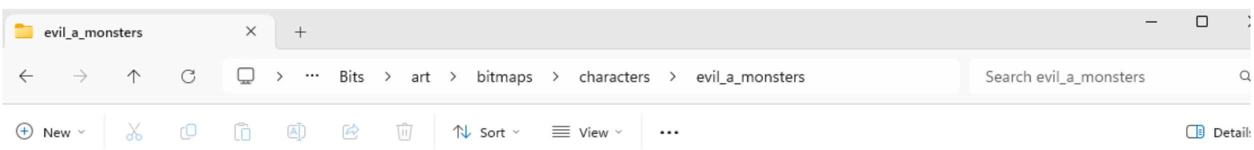



And now we have a .raw file of our pink krug texture.

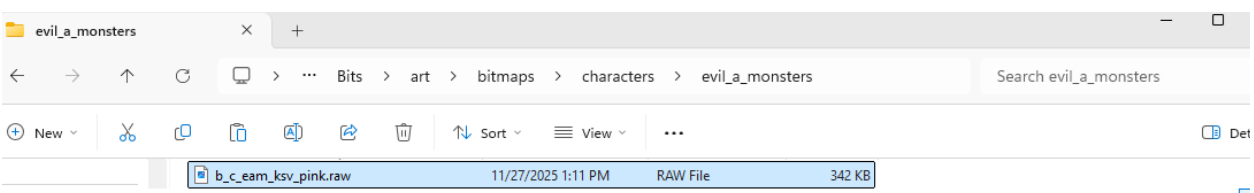



## 8. Place the texture into source files

Now that we have the texture file, we need to place it into our Bits/ folder. Note that within Bits, it needs to respect the vanilla folder structure. That gives us Bits\art\bitmaps\characters\evil_a_monsters.




We place our texture there. Note that it also needs to respect the vanilla naming key. In our case, the prefix b_c_eam must be preserved, whereas the suffix ksv_pink can be changed to anything we like. Preferably something unique that will likely not overwrite vanilla files nor conflict with other mods.




## 9. Reference the texture

Now that the texture exists for our mod and is placed and named correctly, we can tell our new temp_krug template to use this texture. We simply need to replace the vanilla texture name with our new texture name.

```
:template,n:temp_scavenger]

    category_name = "1W_evil_a";
    doc = "temp scavenger";
    specializes = base_krug;
  [actor]
  {
      [skills]
      {
            dexterity = 4, 0;
            intelligence = 3, 0;
            strength = 5, 0;
      }
  }
  [aspect]
  {
        experience_value = 4;
        life = 8;
        mana = 6;
        max_life = 8;
        max_mana = 6;
        scale_base = 0.9;
        selection_indicator_scale = 0.85;
      [textures]
      {
            0 = b_c_eam_ksv_pink;
      }
  }
  [attack]
  {
        damage_max = 2;
        damage_min = 1;
  }
```

## 10. Test

And we're all set. Place your new template into your map and see the visual in the Siege Editor for yourself. There he goes.


Truly a sight for the gods.

# Final remarks – Edge cases

## 10.1. Multiple textures defined

Sometimes, the actor may have multiple textures defined. This is often the case for humanoids wearing clothes, or complex bosses. Take this template for example:

```
[t:template,n:andiemus]
{
    doc = "Andiemus";
    specializes = base_pm_fb;
    [actor]
    {
        portrait_icon = b_gui_ig_i_ic_c_andiemus;
        [skills]
        {
            dexterity       = 4.08, 0, 10;
            intelligence    = 16.8, 0, 10;
            strength        = 3.12, 0, 10;
            nature_magic    = 24,0, 0;
            uber            = 24,0, 0;
        }
    }
    [aspect]
    {
        gold_value = 98880;
        model = m_c_gah_fb_pos_a1;
        [textures]
        {
            0 = b_c_gah_fb_skin_15;
            1 = b_c_pos_a1_064;
        }
    }
    [common]
    {
        screen_name = "Andiemus";
    }
```

We can see two textures here. The texture 0 references texture for his face, hair, feet and hands. The texture 1 is for his clothes – shirt and pants. Multiple textures simply mean the actor consists of multiple parts, and each has a separate texture. For your own derived templates, you can choose to replace all textures, none of them, or just some of them, depending on your taste and the actor.

## 10.2. No textures

Sometimes, an actor might not have a texture defined. This can mean two things.

One is that it may inherit texture from its parent. Templates are object-oriented and support inheritance. For example if we look at seck_archer template, we see no texture (nor other properties) defined.

```
[t:template,n:seck_archer]
{
    category_name = "1W_evil_a";
    doc = "seck archer";
    specializes = seck_archer_base;
}
```

But fear not, as we can see it inherits from the seck_archer_base template. If we look there, we find the texture that belongs to seck_seck archer.

```
[t:template,n:seck_archer_base]
{
    category_name = "1W_evil_a";
    doc = "Seck Archer";
    specializes = base_seck;
    [actor]
    {
        [skills]
        {
            strength =      25, 0;
            intelligence =  19, 0;
            dexterity =     30, 0;
            ranged =        25, 0;
        }
    }
    [aspect]
    {
        experience_value = 73000;
        life = 780;
        max_life = 780;
        mana = 26;
        max_mana = 26;
        model = m_c_eam_sa_pos_1;
        [textures] { 0=b_c_eam_sa; }
    }

    [attack]
    {
        attack_range = 0.5;
        damage_max = 160;
        damage_min = 120;
    }
```

Another case is when actor does not inherit a texture, neither has it defined in its template, such as googore_grub, which inherits from basic textureless generic actor.

```
[t:template,n:googore_grub]
{
    doc = "googore_grub";
    specializes = actor_evil;
    category_name = "1W_evil_d";
    [actor]
    {
        race    = other;

        [skills]
        {
            strength =      8, 0;
            intelligence =  3, 0;
            dexterity =     6, 0;
            melee =         17, 0;
        }
    }
    [aspect]
    {
        model           = m_c_edm_gb_pos_1;
        experience_value    = 2000;
        life                = 260;
        max_life            = 260;
        mana = 6;
        max_mana = 6;
        [voice]
        {
            [die] { * = s_e_die_googore_grub; }
        }
    }
    [attack]
    {
        damage_min      = 125;
        damage_max      = 135;
        attack_range    = 0.5;
    }
```

In this case the texture is defined implicitly. In these cases, the Siege Engine checks the actor's model, and derives the texture from it. In 99% of cases, the texture name in such cases is the model name, with "m" prefix changed to "b", and without the _pos_<x> suffix.

Therefore in this case, since the model is m_c_edm_gb_pos_1, the texture name would be b_c_edm_gb. We can check in the untanked Objects.dsres folder.